MPOO 2027-1 · GRUPO 06

Parcial 1 · correcciones

Versión E del examen del 7 de octubre. Para cada pregunta vienen el enunciado, tu respuesta tal como la enviaste en el portal y su corrección. La calificación no va aquí: la ves con el profesor.

Antes de empezar. Este archivo lo subió el profesor a tu rama. Trae el cambio a tu computadora antes de tu siguiente push, o tu push va a fallar:

```
git checkout entregas_hernandez_emily
git pull --no-rebase
```

Qué tienes que hacer. En hojas, **a mano**, copia cada pregunta y escribe debajo tu respuesta corregida, usando la corrección que viene aquí. Donde la corrección ya te da la salida o el código, no basta con copiarlo: escribe **por qué** sale así, con la tabla de seguimiento de las variables, la causa del error o lo que devuelve tu método en cada renglón de la tabla. Debajo de esas preguntas viene qué escribir. Escanea las hojas o tómales foto, júntalas en un PDF y súbelo a esta misma carpeta, `entregas/hernandez_emily/examen/p1/`, con tu push de siempre.

Pregunta 0.1 · Preguntas abiertas

Pregunta

Contesta con tus palabras, en una a tres oraciones. Puedes apoyarte en un ejemplo pequeño de código
¿Qué significa que un objeto sea una instancia de una clase? Explica si dos instancias deben tener iguales valores en sus atributos.

Tu respuesta

```
que la clase es el molde de este objeto
```

Corrección. Reconoce que la clase es el molde del objeto, pero no explica que la instancia es un objeto concreto creado a partir de ella. Tampoco contesta si dos instancias deben tener los mismos valores: no tienen que tenerlos, porque cada una guarda sus propios valores de instancia.

Pregunta 0.2 · Preguntas abiertas

Pregunta

¿Qué diferencia hay entre un parámetro y un argumento de un método? Ilústrala con una llamada sencilla.

Tu respuesta

```
parametro es la variable en la cual se cobnsagra el metodo y el argumento es el valor de esta
```

Corrección. La distinción básica es correcta: el parámetro es una variable y el argumento es el valor que recibe. Faltó precisar que el parámetro se declara en la cabecera y el argumento se pasa en la llamada, y faltó el ejemplo pedido, como int cantidad en cargar(int cantidad) frente al 3 en cargar(3).

Pregunta 0.3 · Preguntas abiertas

Pregunta

¿Qué es el ámbito de una variable local? Da un ejemplo de dónde se puede usar y dónde deja de estar disponible.

Tu respuesta

Sin respuesta.

> **Corrección.** Sin respuesta.

Pregunta 0.4 · Preguntas abiertas

Pregunta

¿Qué guarda una variable de tipo arreglo? Explica qué ocurre al asignarla a otra variable del mismo tipo.

Tu respuesta

```
guarda un arrehlo entero. Al asignarla , la nueva variable apunta en direccion a este arreglo,
es posible cambiar sus elementos desde esta variable. NO se crea otro arreglo, se crea una
direccion o referencia que apunta a este arreglo original.
```

> **Corrección.** Correcta: la variable guarda una referencia al arreglo, y al asignarla las dos variables apuntan al mismo arreglo, sin copiarlo, así que un cambio hecho por una se ve en la otra.

Pregunta 0.5 · Preguntas abiertas

Pregunta

¿Qué hace break dentro de un case de un switch clásico? Explica qué puede ocurrir si falta.

Tu respuesta

```
detiene el procedimiento, por esto el switch se escribe en orden. Si se escribe un caso y al
finalizar ese caso no se pone un break, se ejecuta el siiguiente caso y puede imprimir lo qur
nos asigna el segundo caso.
```

> **Corrección.** Explica bien que, si falta break, se ejecuta también el case siguiente. Decir que break 'detiene el procedimiento' es impreciso: break solo sale del switch y la ejecución sigue con la instrucción que viene después de él.

R1 · ¿Qué imprime?

Pregunta

Escribe la salida exacta, una línea por renglón. El fragmento está en un main válido.

```
Integer original = 7;
Integer copia = original;
int vuelta = 0;
do {
    original = original + 2;
    System.out.println(original / 2);
    vuelta++;
```

```
} while (vuelta < 3);
String texto = "E" + copia + 1;
System.out.println(texto);
System.out.println((double) (original / 2));
```

Tu respuesta

```
4
5
6
E 14
6.0
```

Corrección. Las líneas 4, 5, 6 y 6.0 son correctas. La cuarta línea debía ser E71: "E" + copia + 1 concatena de izquierda a derecha y copia vale 7. El prefijo E es correcto.

Qué escribir a mano. La salida ya la tienes en la corrección. Escribe por qué sale así: una tabla de seguimiento con el valor de cada variable en cada vuelta y en cada println.

R2 a) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
String codigo = null;
boolean igual = codigo.equals("R3");
System.out.println(igual);
```

Tu respuesta

Clasificación

```
Compila pero truena
```

Causa o salida

```
compila por que es valido haer que una string sea null, pero como intenta comparar una variable R3 que nisiquiuera esta declarada, es decir, no existe, hace que nuestro programa truene.
```

Corrección. La clasificación es correcta: compila pero truena. La causa no lo es: "R3" es un literal String y no una variable sin declarar. El programa truena porque se llama a equals sobre una referencia null (NullPointerException).

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 b) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
double voltaje = 7.6;
int lectura = voltaje;
System.out.println(lectura);
```

Tu respuesta

Clasificación

Funciona

Causa o salida

7.6

Corrección. Incorrecto: el fragmento no compila, porque Java no convierte de forma implícita un double en int (asignar 7.6 a un int exige un cast explícito).

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 c) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
Integer reserva = 4;
int nueva = reserva + 3;
System.out.println(nueva);
```

Tu respuesta

Clasificación

Compila pero truena

Causa o salida

no es posible sumar un integer con un int porque uno es primitivo

Corrección. Incorrecto: el fragmento funciona e imprime 7, porque Java desempaqueta el Integer en un int para hacer la suma.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R3 · Modelado: base, reto y justificación

Pregunta

Cada banda nace vacía con capacidad propia fija. El límite de seguridad común es 12 piezas. agregarPieza() suma una y consultarPiezas() responde cuántas hay. Supón que solo se llama agregarPieza cuando ambos límites lo permiten.

Base. Declara al menos dos atributos de instancia con tipo, uno static y una constante final (¿también static?). Completa constructor y dos métodos: cambio y consulta.

Reto. ¿Cuál es final de instancia y cuál static final: capacidad o límite de seguridad?

Justifica. Una línea por decisión: tipo, instancia/clase, final o mutable, constructor y métodos.

Cuerpos de 1 a 3 líneas, sin estructuras de control ni main. Hay más de un modelo correcto. Un elemento sin justificar vale la mitad.

Tu respuesta

Clase

```
Base
int piezas;
static int capacidad;
final int capacidadcomun = 12;

Reto:

:0
```

Justificaciones

Sin respuesta.

> **Corrección.** Piezas como int de instancia es correcto, y el límite 12 aparece como final, pero capacidad no puede ser static porque cada banda tiene la suya: debía ser final de instancia. El límite común debía ser static final. Faltan el constructor (this.capacidad = capacidad; piezas = 0), los métodos agregarPieza y consultarPiezas, la respuesta al reto y todas las justificaciones; cada elemento necesita su justificación.

R4 · Código con repetición

Pregunta

Un equipo parte con carga unidades. En cada paso suma aporte. Devuelve cuántos pasos necesita para alcanzar o superar meta usando while. Si ya llegó, devuelve 0. Carga y meta están entre 0 y 10000; aporte está entre 1 y 100. No uses una fórmula de división.

```
static int pasosCarga(int carga, int aporte, int meta)
```

Ejemplos (argumentos → retorno):

5, 4, 16 → 3 · 16, 4, 16 → 0 · 20, 4, 16 → 0

Escribe el método completo. No imprimas ni leas del teclado.

Tu respuesta

```
public class CagaUnidades{

public static void main(String[] arrgs){

int paso = 1;

int aporte = 100; //debe estar entre 0 y 100. como no nos daban un valor, yo decidi asuumirlo
o mejor dicho, darle uno. al igual que las demas variables, y que solo nos daba un rango.

int meta= 10000;

int carga = 0;
```

```
int suma = 0

int faltante = 10000;

while(suma<10000){

suma = aporte + suma;

faltante = (meta – (paso* aporte);
faltante = faltante/paso // aqui vi necesario dividir poir que no encontre otra forma de
cslculsar los pasos faltantes.

paso ++;

System.out.println("faltan" + faltante + " pasos para llegar a la meta")

}




}
}
```

Corrección. Escribió un main con valores fijos en lugar del método static int pasosCarga(int carga, int aporte, int meta). Usa while, acumula aporte y cuenta pasos, pero empieza la suma en 0 sin usar carga y empieza el contador en 1. Además imprime, aplica una división que el enunciado prohíbe y no devuelve nada. Lo correcto era int pasos = 0; while (carga < meta) { carga += aporte; pasos++; } return pasos;.

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.

R5 · Código con selección

Pregunta

Decide si un banco de pruebas puede arrancar. Recibes el porcentaje de batería y si la tapa está cerrada. Aplica la primera regla que corresponda, en el orden de la tabla. Usa if / else-if / else, con && y ||. No imprimas ni leas del teclado.

```
static String permiso(int bateria, boolean tapaCerrada)
```

Regla o valor recibido	Texto devuelto
Batería menor que 0 o mayor que 100	"INVALIDA"
Tapa cerrada y batería al menos 60	"NORMAL"
Tapa cerrada y batería al menos 20	"AHORRO"
Cualquier otro caso	"BLOQUEADO"

Tu respuesta

```
public class Banco{

public static void main(String[] arrgs){


Scanner sc = //aqui algo// Scanner(System.in);

System.out.println("Bateria:")

int bateria = sc.nextInt();

boolean tapa;

System.out.println("La tapa esta cerrada?( selecciona 1 o 2):
1. si
2. no")

swich{
opcion 1:
boolean tapa = true;
breack;
opcion 2:
boolean tapa = false;
break;
defaul:
System.outprintln("opcion no valida");
breack;
}

if(bateria<0 ||tapa>100){
System.out.println("INVALIDA");
}
else if(tapa && tapa>=60){
System.out.println("NORMAL");

}
else if(tapa && tapa>=20){
System.out.println("AHORRO");
}
default{
System.out.println("BLOQUEADO");

}

}
```

```
}
```

Corrección. El orden de las reglas y el uso de || y && siguen la tabla, y las fronteras 60 y 20 son correctas. Hay cuatro fallas: escribió un main que lee del teclado e imprime en lugar del método static String permiso(int bateria, boolean tapaCerrada) con return; compara tapa contra números donde debía usar bateria (bateria > 100, tapaCerrada && bateria >= 60); usa default en lugar de else; y la sintaxis tiene muchos errores (swich, breack, puntos y coma faltantes, tapa declarada dos veces).

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.